



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

15. Fragestunde



Komplexität



O-Notation

- Definition

- Seien $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ zwei Funktionen. Dann ist

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

- Man sagt: f hat eine Komplexität von $O(g)$.
 - f wächst höchstens so schnell wie g .
 - Statt $f \in O(g)$ oft auch $f=O(g)$ geschrieben.
 - Landau Symbole $O, o, \Omega, \omega, \Theta$



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$f \in O(g)$$

f wächst höchstens so schnell wie g



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$\exists n_0 \in \mathbb{N}$$

Startpunkt n_0

Wir schauen auf große n , was vorher passiert ist egal.



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$\exists c \in \mathbb{R}^+$$

Positive Konstante mit der wir g multiplizieren dürfen.

Wir dürfen g vergrößern, um f zu überdecken.



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$\forall n \in \mathbb{N}$$

Für alle n Element der natürlichen Zahlen gilt



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

$$n > n_0$$

Ab diesem Punkt gilt die Ungleichung



O-Notation

$$f \in O(g) \Leftrightarrow \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \in \mathbb{N}: \\ n > n_0 \Rightarrow f(n) \leq c \cdot g(n)$$

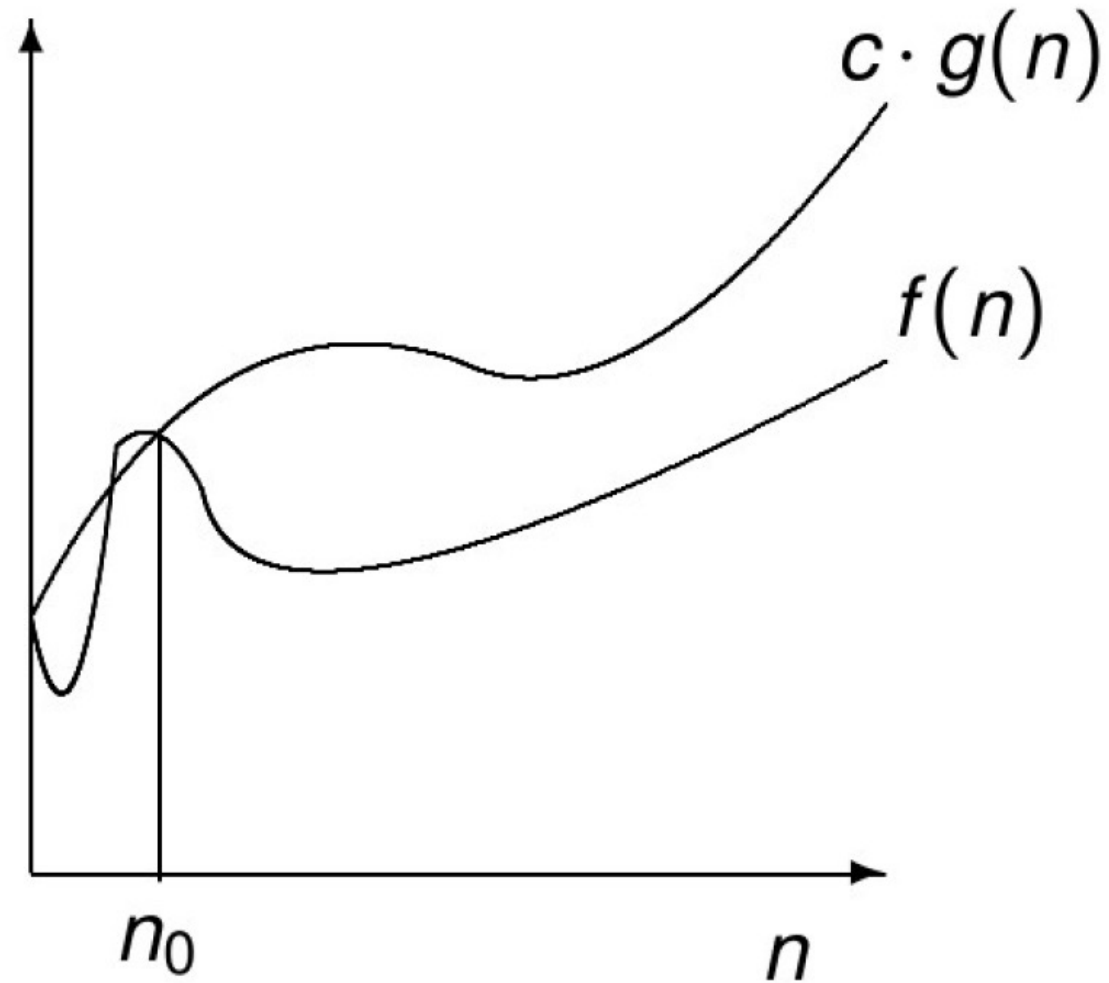
$$f(n) \leq c \cdot g(n)$$

Die Funktion f liegt unter einer skalierten Version von g



O-Notation

- $f \in O(g)$





Beispiele O-Notation

- $n+7 \in O(n)$
 - Wähle $n_0 = 7$ und $c=2$, dann ist für $n>7$: $n+7 \leq n+n=2*n$
- $2^{20}n + 2^{30} \in O(n)$
 - Wähle $n_0 = 2^{10}$ und $c=2^{21}$, dann ist für $n>n_0$:
 - $2^{20}n + 2^{30} = 2^{20}(n + 2^{10}) \leq 2^{20}(n + n) = c*n$
- $f+g \in O(f)$ falls $g \in O(f)$
 - Aus $g \in O(f)$ folgt: es gibt n_{0g} und c_g . Wähle nun $n_0=n_{0g}$ und $c = c_g+1$.
Dann gilt $f+g \leq f+c_g*f = c*f$
- $f*g \notin O(f)$ falls $g \notin O(1)$.
 - Angenommen, $f*g \in O(f)$, dann gibt es n_0 und c mit
 - $f*g \leq c*f$. Damit gilt $g \leq c$ für alle $n>n_0$. Also $g \in O(1)$.



O-Notation praktisch

- Weglassen von multiplikativen Konstanten
 - $2 \cdot n \in O(n)$
- Weglassen von additiven Konstanten (Startup-Kosten)
 - $n + c \in O(n)$
- Beschränkung auf höchsten Exponenten
 - $n^2 + n \in O(n^2)$
- Basis des Logarithmus ist unerheblich
 - $\log_2 n \in O(\log n)$
 - $\log_b n = \log_a n \cdot \log_b a$



Typische O-Notation

- $O(f)$ ist nur Obergrenze
 - f sollte einfach sein
 - Keine konstanten Faktoren
 - Keine langsam wachsenden Summanden
 - Möglichst dicht!
 - Teilweise sehr langsam wachsende Faktoren ignoriert:
 - $O(n^2 \log(\log n))$
- $O(1)$
 - $O(\log n)$
 - $O(\sqrt{n})$
 - $O(n)$
 - $O(n \log n)$
 - $O(n^2)$
 - $O(n^k)$
 - $O(2^n)$
 - ...



Typische Zeitkomplexitäten

- **$O(1)$ Konstante Komplexität:** Einige Suchverfahren für Tabellen (Hashing)
- **$O(\log n)$ Logarithmische Komplexität:** Sehr günstig, da die Laufzeit erheblich schwächer wächst als n . Binäre Suche
- **$O(n)$ Lineare Komplexität:** Immer noch günstig. Auf jedes der n Eingabeelemente entfällt eine feste Verarbeitungszeit.
- **$O(n \log n)$ Leicht überlineare Komplexität:** Tritt oft auf, wenn ein Problem fortgesetzt in Teilprobleme zerlegt wird, die unabhängig voneinander gelöst werden. Beispiel: Sortieren
- **Quadratische Komplexität:** Ungünstig. Zwei geschachtelte Schleifen, die auf alle n Daten angewandt werden.
- **Kubische Komplexität:** Sehr ungünstig und nur auf kleine n anwendbar.
- **Exponentielle Komplexität:** Katastrophal für große n . Tritt auf, wenn zur Lösung des Problems alle Möglichkeiten exhaustiv geprüft werden müssen. Viele Optimierungsprobleme!



Aufgabe 1

Bestimmen Sie die engste Θ -Schranke:

$$100 \cdot \sqrt{n} + 0,01 \cdot n \in \Theta(?)$$



Aufgabe 2

Bestimmen Sie die engste Θ -Schranke:

$$2^n + n^{100} + n! \in \Theta(?)$$



Aufgabe 3

Bestimmen Sie die engste Θ -Schranke:

$$n^{\log_2 n} + 2^n \in \Theta(?)$$



Aufgabe 4

Bestimmen Sie die engste Θ -Schranke:

$$2^{(n+10)} + 5 \cdot 2^n \in \Theta(?)$$



Aufgabe 5

Bestimmen Sie die engste Θ -Schranke:

$$2^{(n+10)} + 5 \cdot 2^n + 3x^2 \in \Theta(?)$$

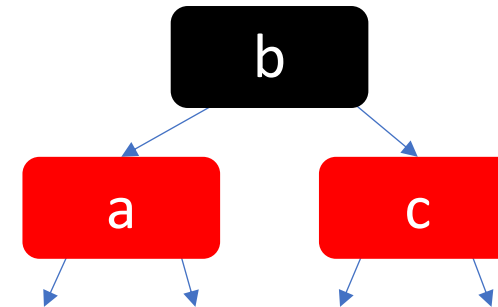
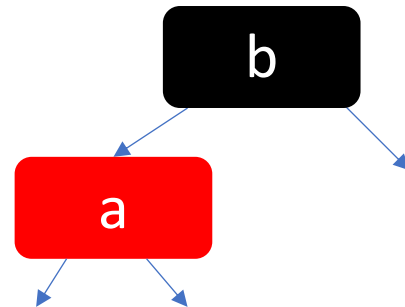


Rot-Schwarz-Bäume



Definition Rot-Schwarz Bäume

- **Rot-Schwarz-Bäume** sind binäre Suchbäume, die die folgenden Kriterien, die sogenannten Rot-Schwarz-Eigenschaften, erfüllen:
 - Jeder Knoten ist entweder rot oder schwarz.
 - Jeder Null-Knoten ist per Definition schwarz.
 - Die Kinder jedes roten Knotens sind schwarz.
 - Für jeden Knoten k gilt: Jeder Pfad von k zu einem Blatt enthält die gleiche Anzahl schwarzer Knoten.





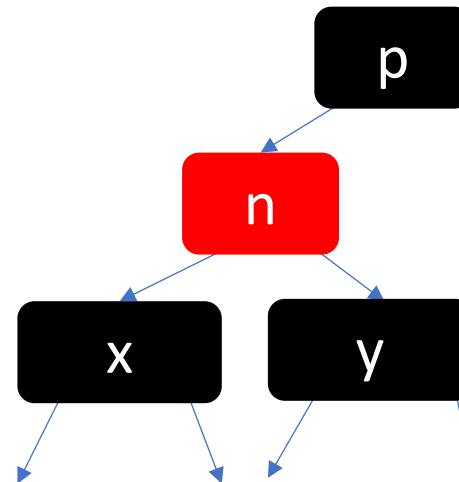
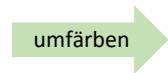
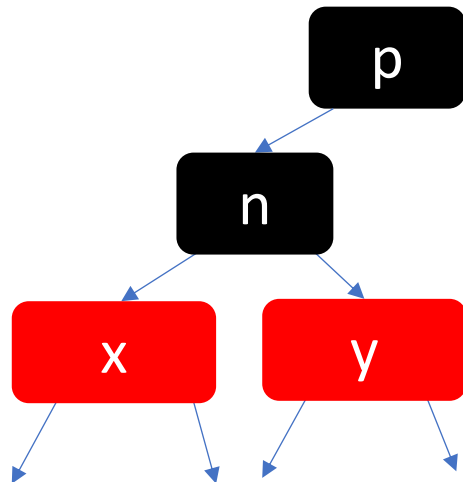
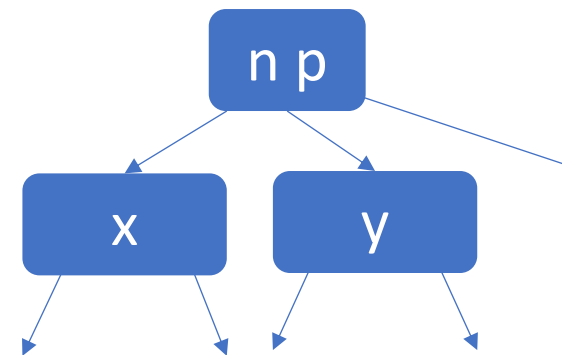
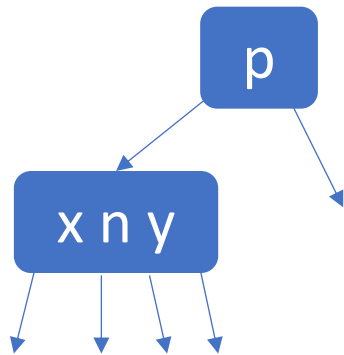
Einfügen in Rot-Schwarz Bäumen

- Splitten der 4-Knoten: top-down, nichtrekursiv
- zwei Hauptfälle unterscheiden
 - 1. der 4-Knoten hängt an einem 2-Knoten
 - 2. der 4-Knoten hängt an einem 3-Knoten
- Unterfälle (2.):
 - A Teilbaum hängt links
 - B Teilbaum hängt in der Mitte
 - C Teilbaum hängt rechts
- Bezeichnungen:
 - n aktuell betrachteter Knoten (node)
 - p dessen Elternknoten (parent)
 - g dessen Elternknoten von p (grandparent)



Splitten in Rot-Schwarz Bäumen: Fall 1

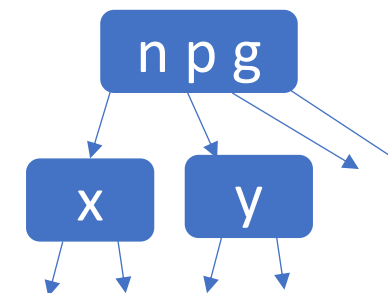
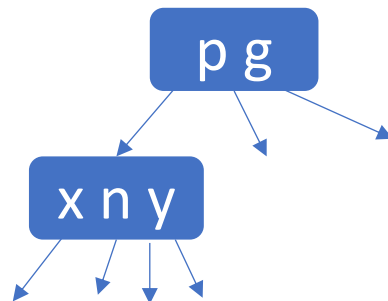
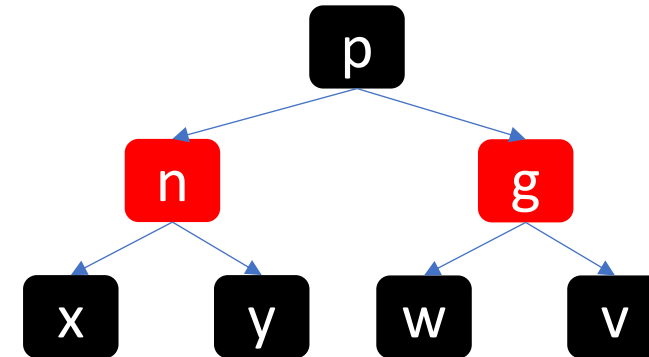
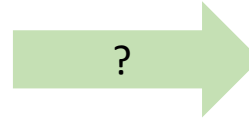
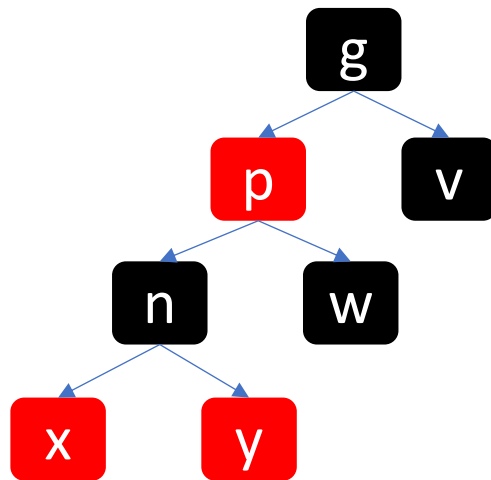
- Fall 1: 4 Knoten soll gesplittet werden und hängt an 2 Knoten





Splitten in Rot-Schwarz Bäumen: Fall 2A

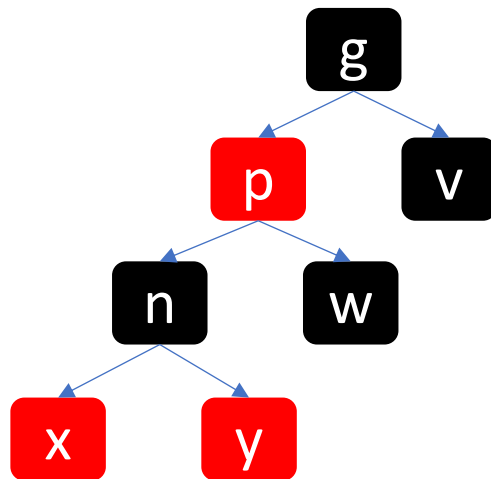
- Fall 2A: 4 Knoten soll gesplittet werden und hängt an 3 Knoten links außen



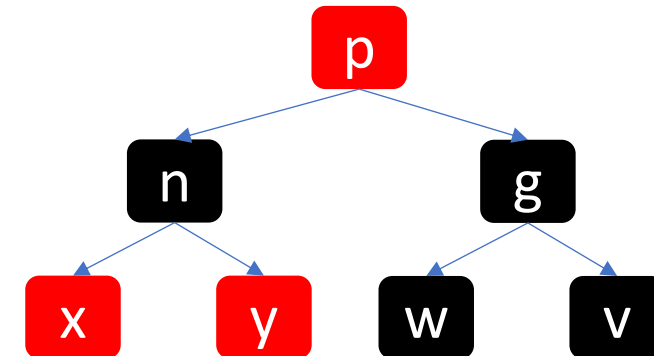


Splitten in Rot-Schwarz Bäumen: Fall 2A

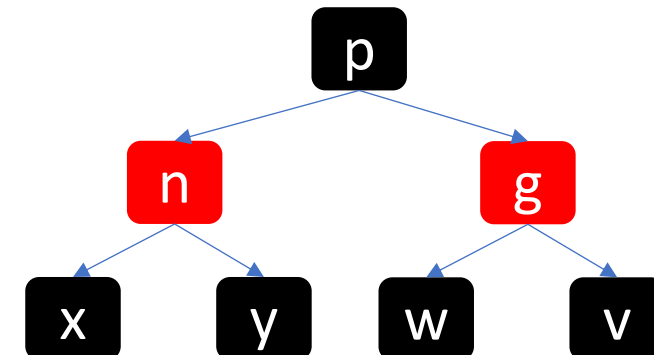
- Fall 2A: 4 Knoten soll gesplittet werden und hängt an 3 Knoten links außen



Rotation



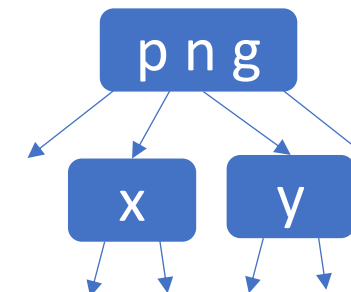
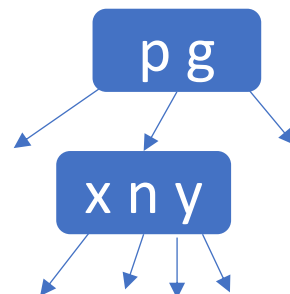
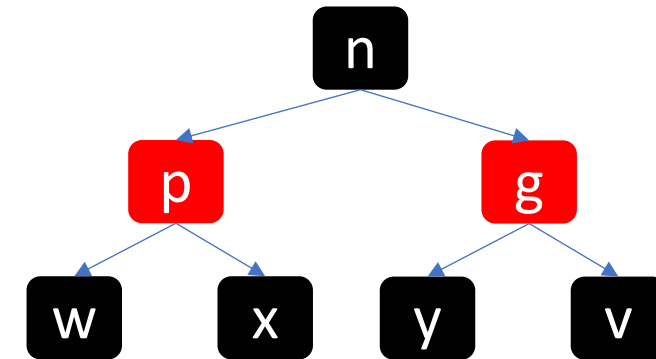
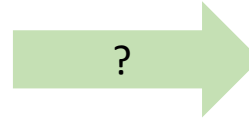
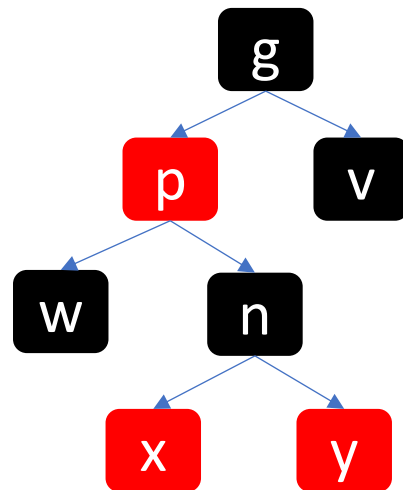
umfärben





Splitten in Rot-Schwarz Bäumen: Fall 2B

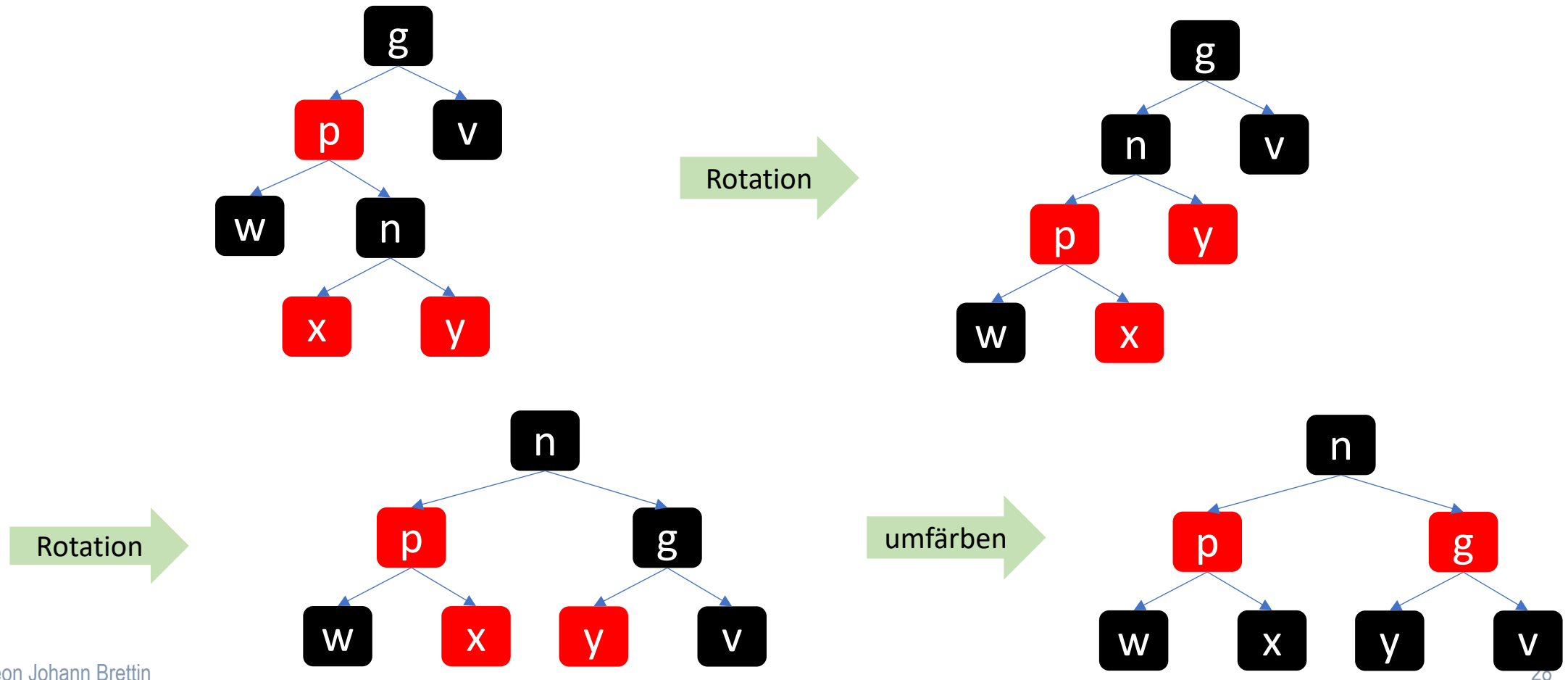
- Fall 2B: 4 Knoten soll gesplittet werden und hängt an 3 Knoten in der Mitte





Splitten in Rot-Schwarz Bäumen: Fall 2B

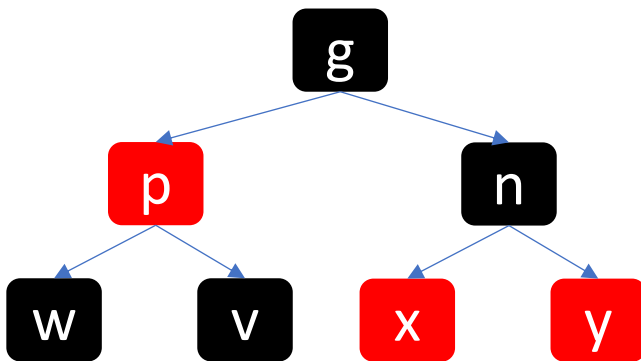
- Fall 2B: 4 Knoten soll gesplittet werden und hängt an 3 Knoten in der Mitte



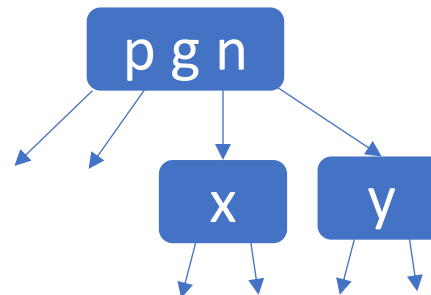
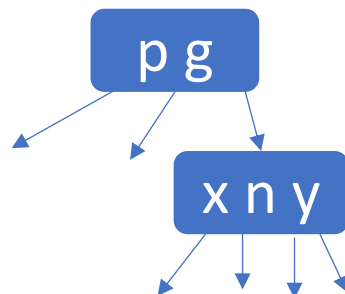
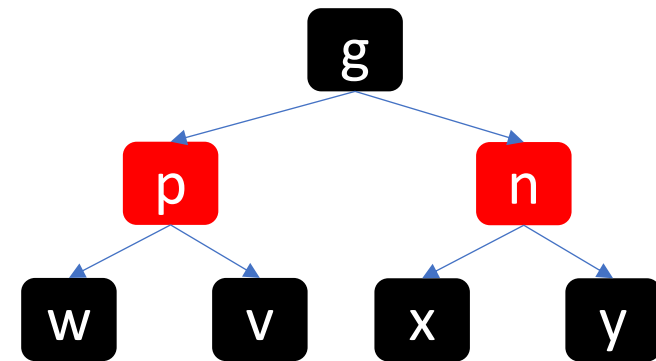


Splitten in Rot-Schwarz Bäumen: Fall 2C

- Fall 2C: 4 Knoten soll gesplittet werden und hängt an 3 Knoten rechts außen



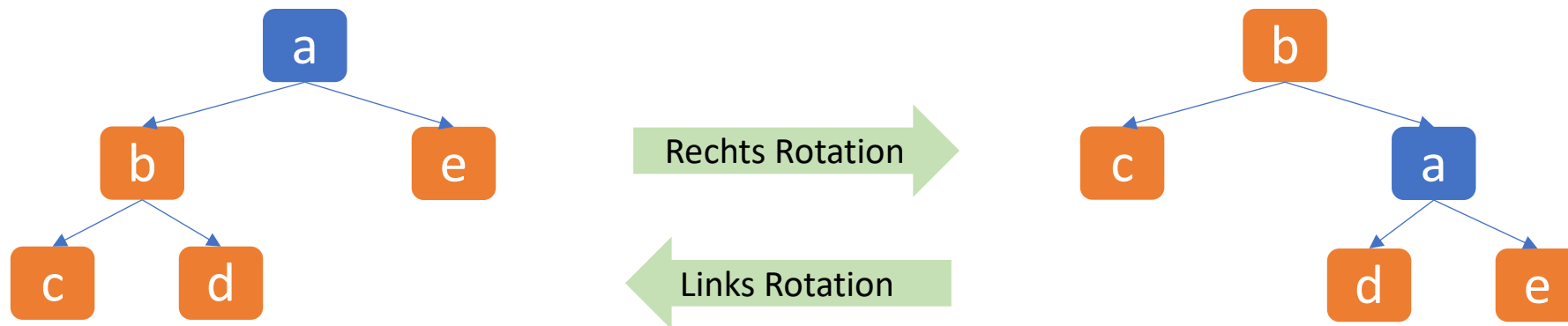
umfärben





Rotation in Suchbäumen

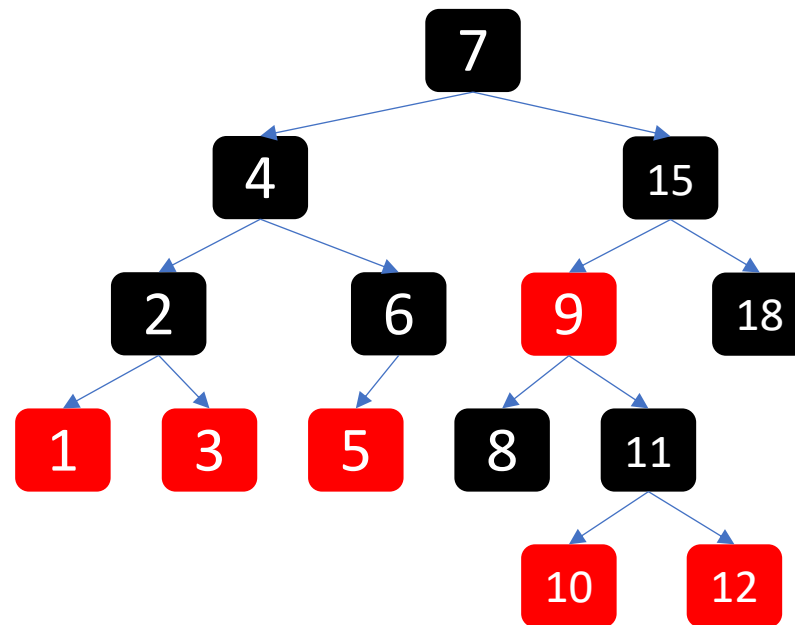
- Realisierung des Splittens von Knoten durch die Operationen Umfärben und Rotation.
- Rotationen verletzen die Suchbaumeigenschaft nicht.
- Komplexität: $O(1)$





Beispiel: Einfügen von 13

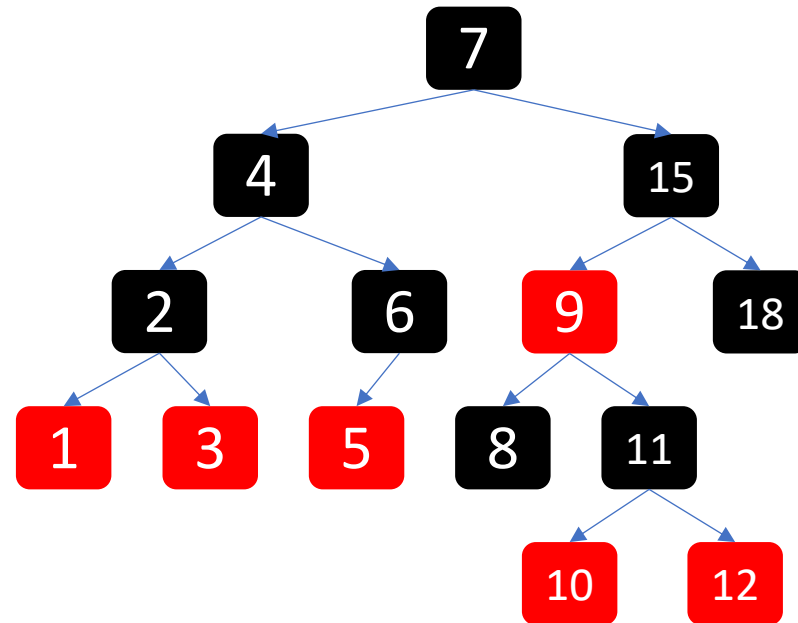
- Top-down: Einfügeposition suchen, dabei alle 4 Knoten splitten.





Beispiel: Einfügen von 13

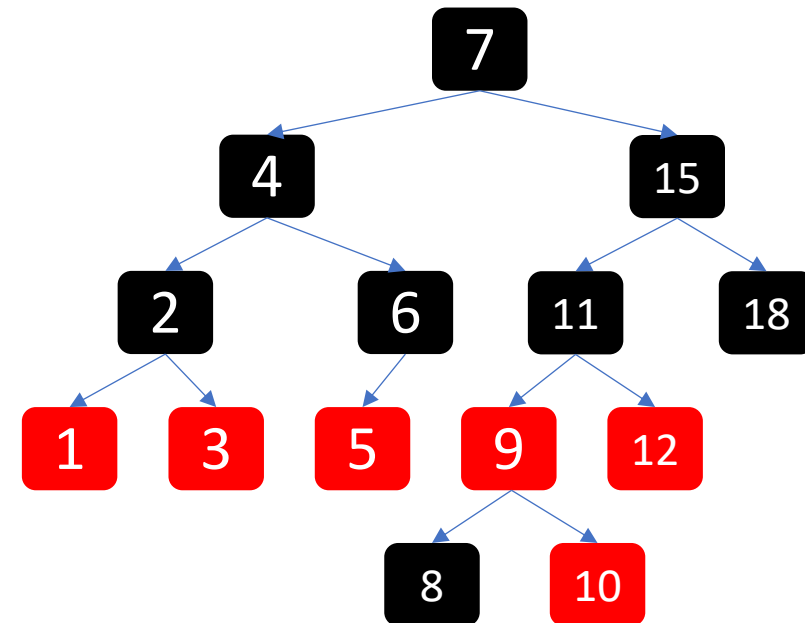
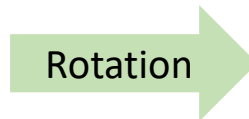
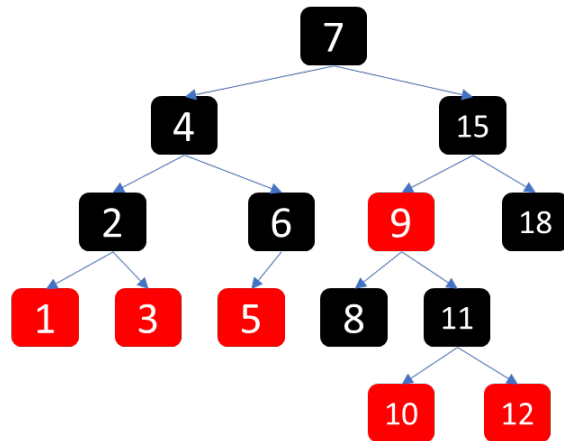
- 11 ist ein 4 Knoten, Fall 2B





Beispiel: Einfügen von 13

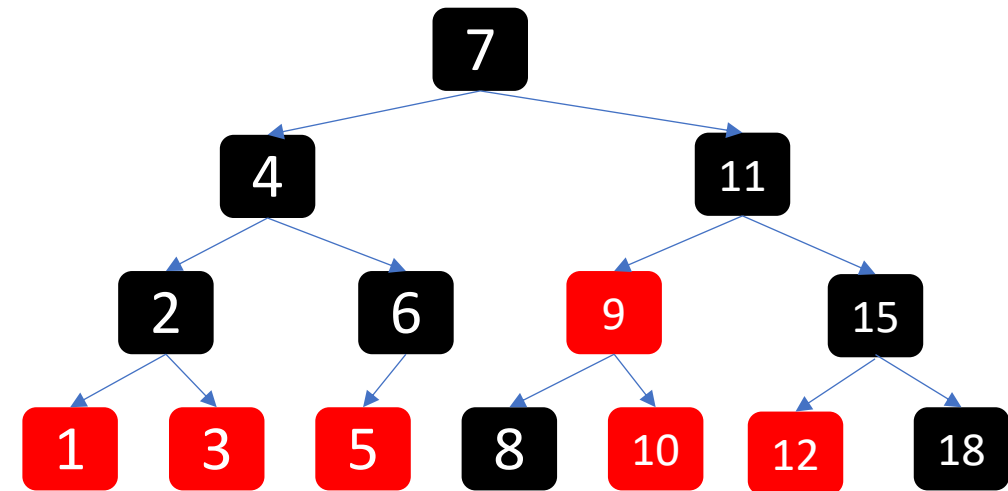
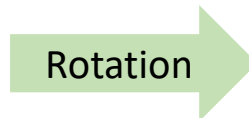
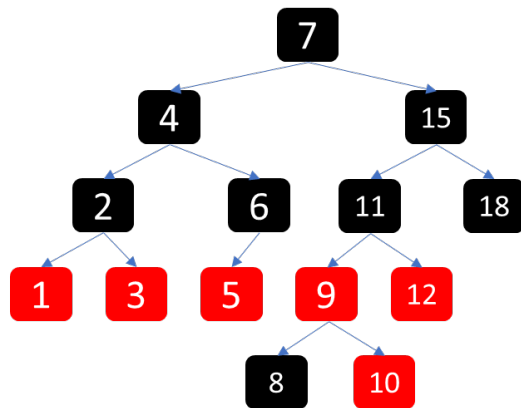
- 2-mal rotieren
- Umfärben





Beispiel: Einfügen von 13

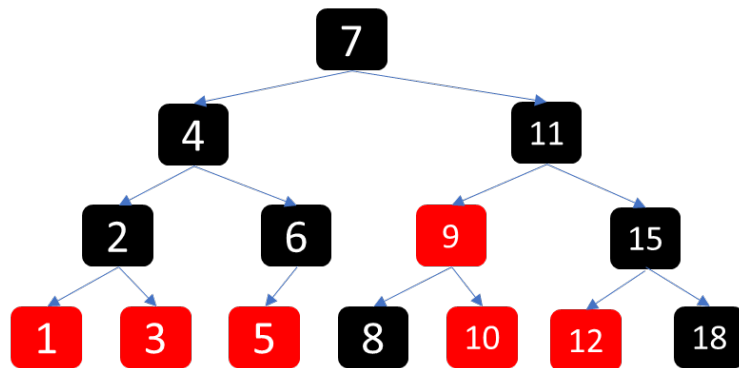
- 2-mal rotieren
- Umfärben



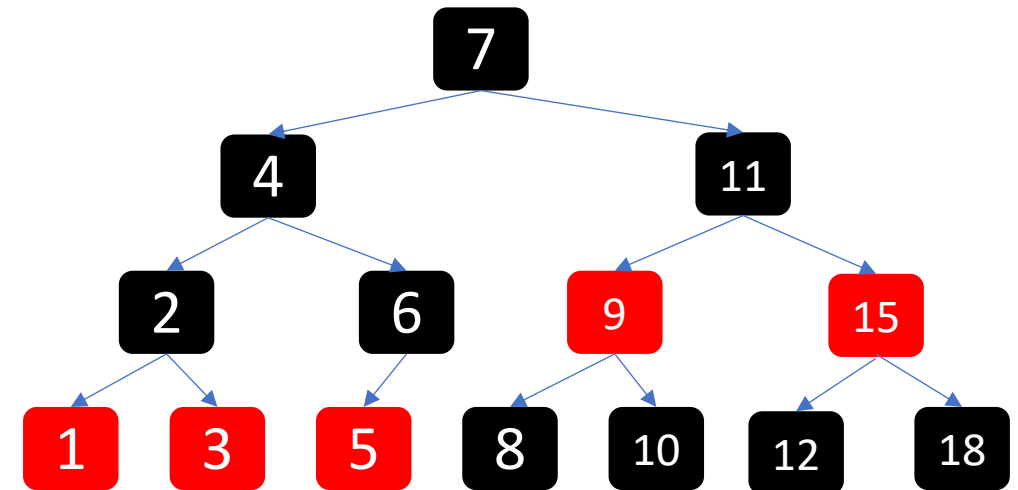


Beispiel: Einfügen von 13

- 2-mal rotieren
- **Umfärben**



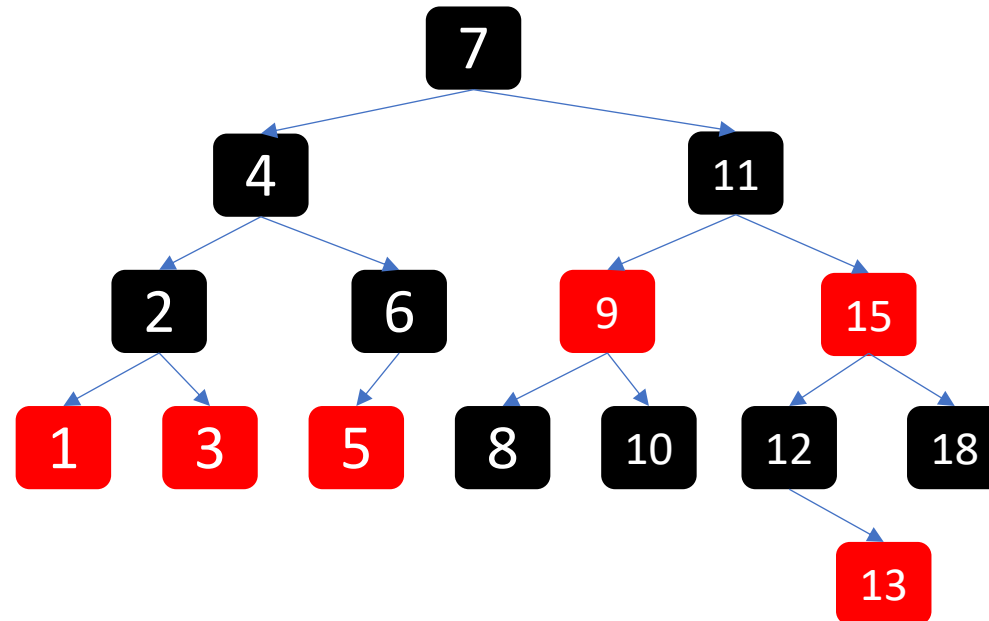
umfärben





Beispiel: Einfügen von 13

- Weiter nach Position für 13 suchen!
- 13 Einfügen



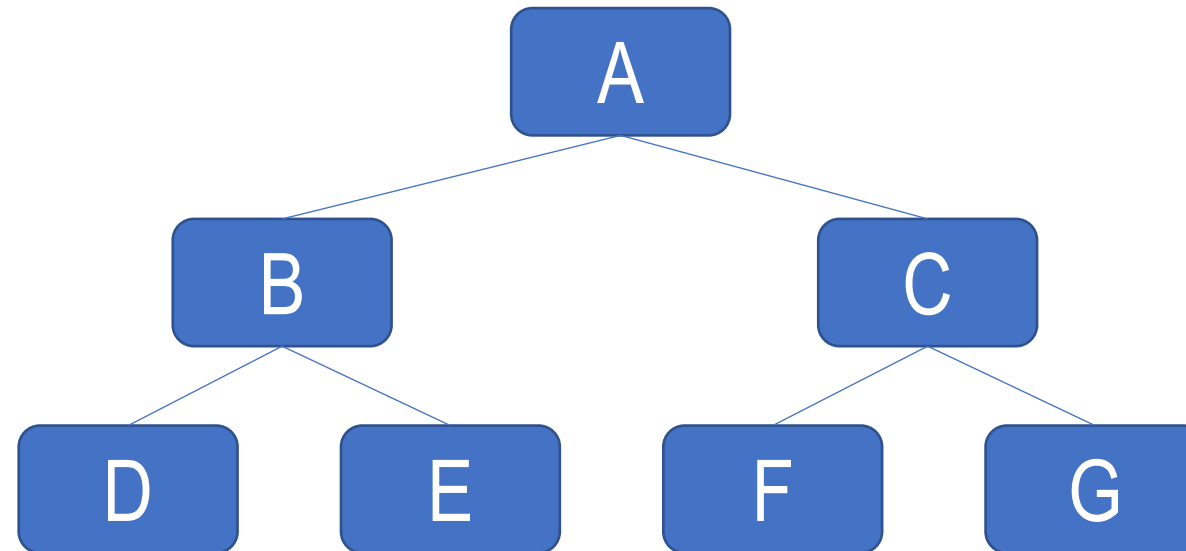


Baumtraversierung



Traversierung von Bäumen

- Traversierung eines Baums ist das systematische Durchlaufen eines Baums
- Dabei werden alle Knoten besucht und können systematisch abgearbeitet werden
- Dafür lassen sich verschiedene Strategien finden.

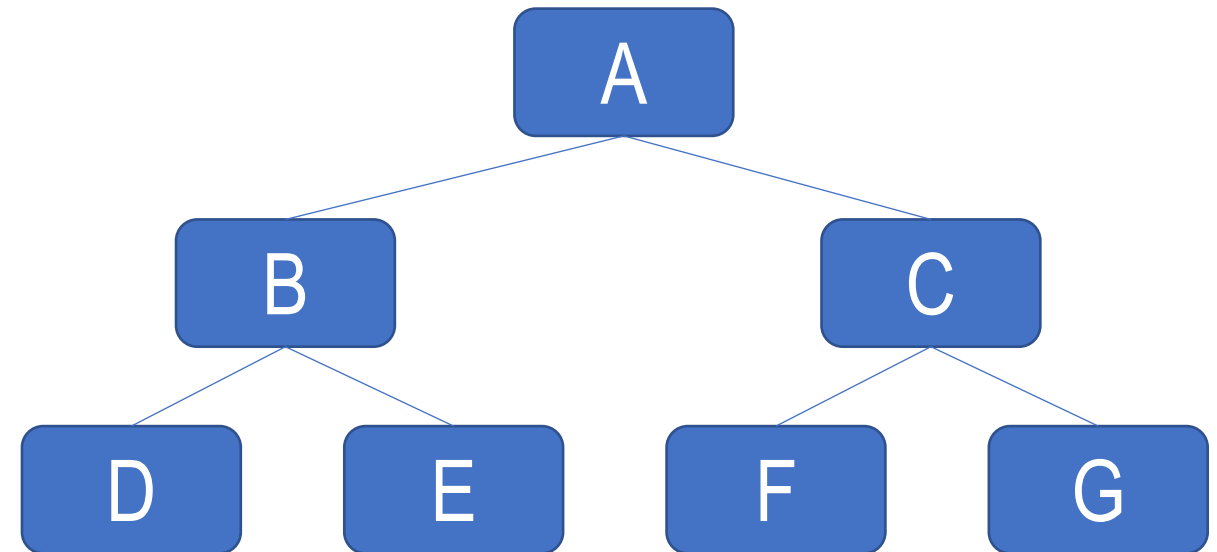




Inorder

- Inorder-Traversierung:

- linker Teilbaum
- aktueller Knoten
- rechter Teilbaum



- D – B – E – A – F – C – G



Algorithmus Inorder-Traversierung

```
algorithm Inorder(k)
```

```
    Eingabe: Knoten k eines Binärbaums
```

```
    if k = null return
```

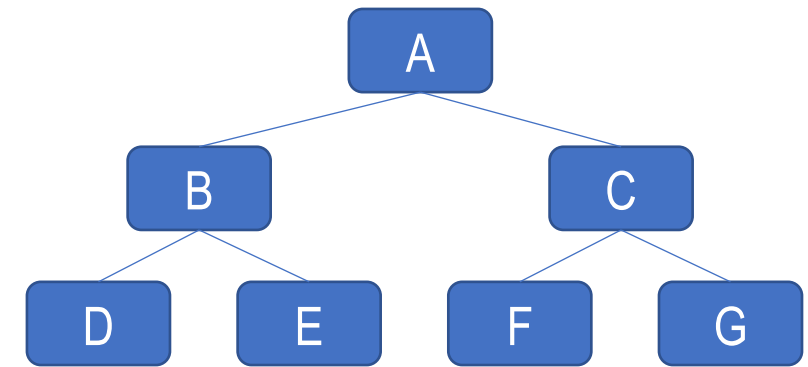
```
    /* besuche linken Teilbaum */
```

```
    Inorder(k.left);
```

```
    Verarbeite k.elem;
```

```
    /* besuche rechten Teilbaum */
```

```
    Inorder(k.right);
```

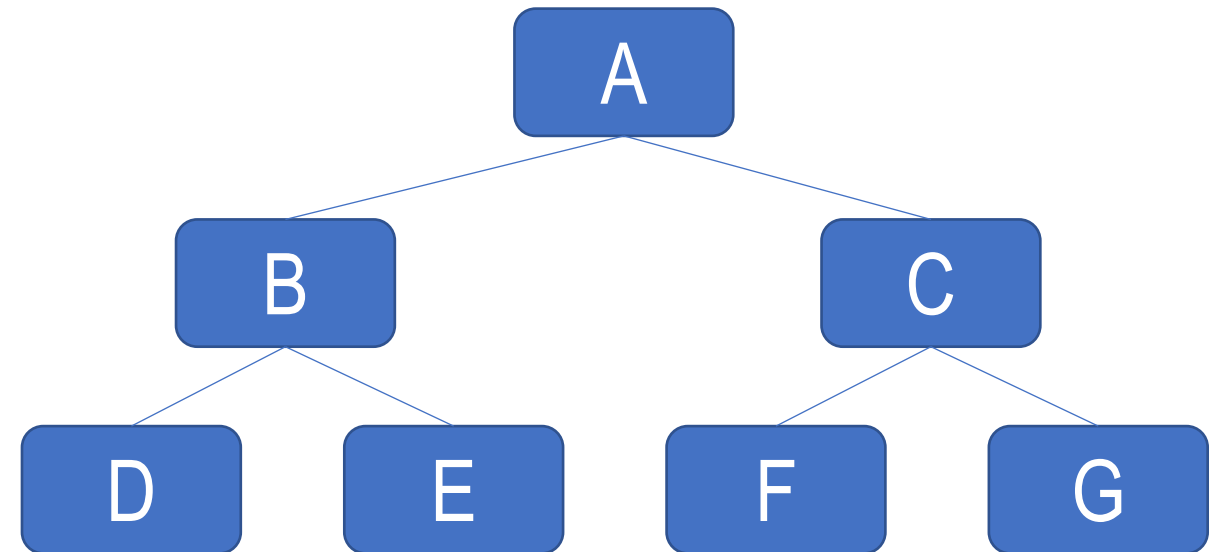




Preorder

- Preorder-Traversierung:

- aktueller Knoten
- linker Teilbaum
- rechter Teilbaum

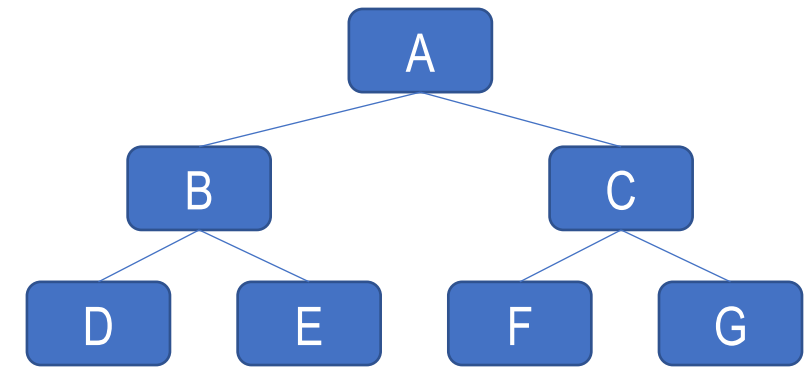


- A – B – D – E – C – F – G



Algorithmus Preorder-Traversierung

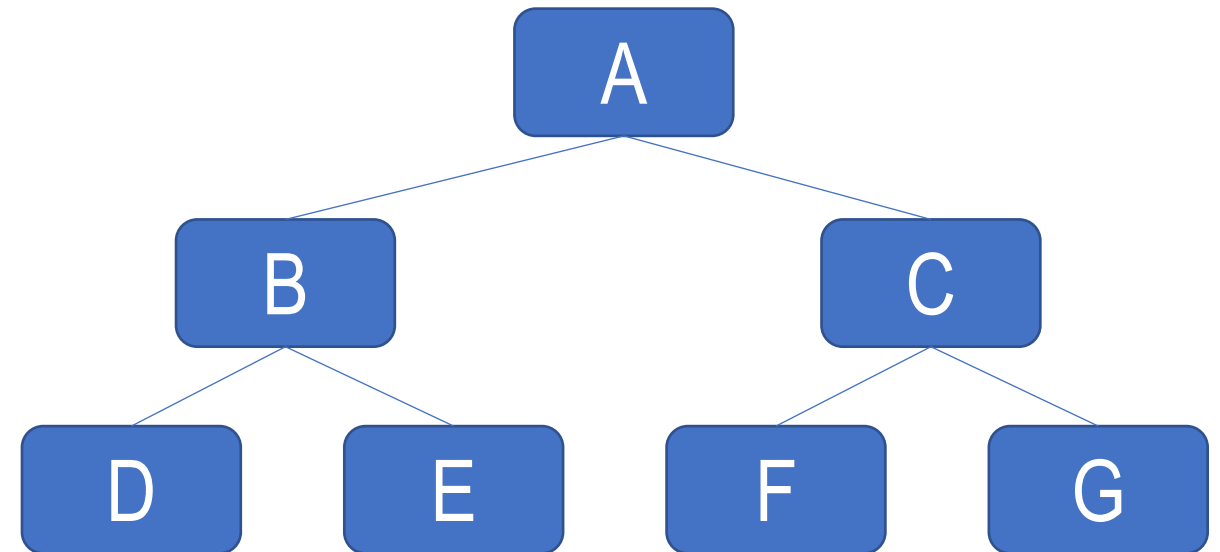
```
algorithm Preorder(k)
  Eingabe: Knoten k eines Binärbaums
  if k = null return
  Verarbeite k.elem;
  /* besuche linken Teilbaum */
  Preorder(k.left);
  /* besuche rechten Teilbaum */
  Preorder(k.right);
```





Postorder

- Postorder-Traversierung:
 - linker Teilbaum
 - rechter Teilbaum
 - aktueller Knoten

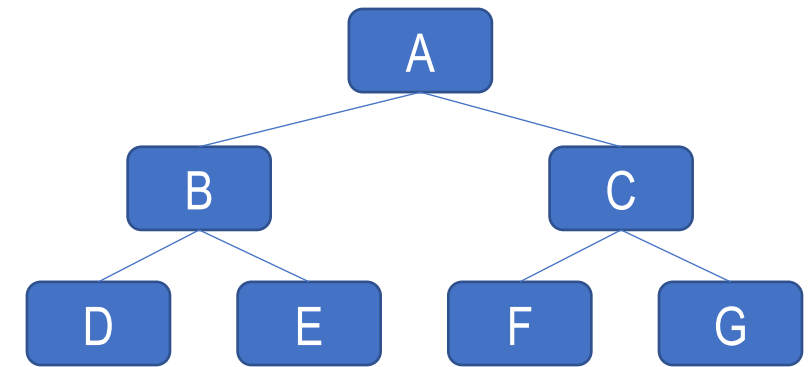


- D – E – B – F – G – C – A



Algorithmus Postorder-Traversierung

```
algorithm Postorder(k)
  Eingabe: Knoten k eines Binärbaums
  if k = null return
  /* besuche linken Teilbaum */
  Postorder(k.left);
  /* besuche rechten Teilbaum */
  Postorder(k.right);
  Verarbeite k.elem;
```





Verallgemeinerung Traversierung

algorithm **Visit**(k)

Eingabe: Knoten k eines Binärbaums

if k = null **return**

Verarbeite k.elem; // pre

/* besuche linken Teilbaum */

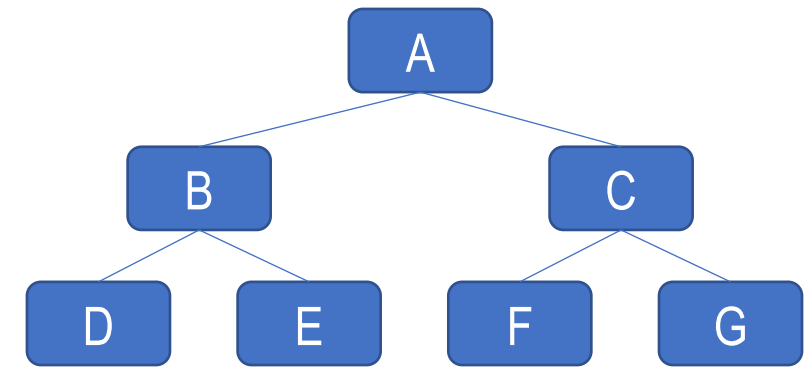
Visit(k.left);

Verarbeite k.elem; // in

/* besuche rechten Teilbaum */

Visit(k.right);

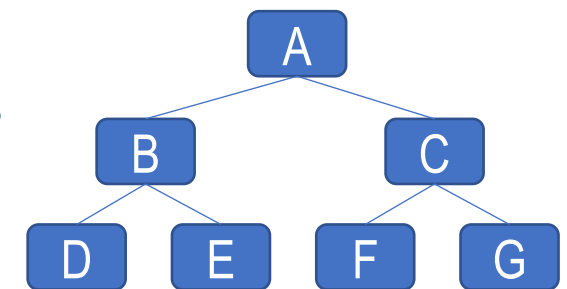
Verarbeite k.elem; // post





Levelorder Durchlauf

- Entspricht der **Breitensuche** im Baum, d.h. auf jedem Niveau des Baums werden erst alle Knoten besucht (beginnend mit der Wurzel), bevor auf das nächste Niveau gewechselt wird.
- Ausgabe der Knoten geordnet nach Baumebene, beginnend bei der Wurzel und Ordnung innerhalb der Baumebene, beginnend bei kleinstem Element
- Algorithmus ist im Gegensatz zu bisherigen Strategien **nicht rekursiv**.
- Noch zu verarbeitende Knoten werden in einer **Warteschlange** abgelegt.
- Der Baum wird durchlaufen, indem der nächste zu besuchende Knoten aus der Warteschlange entnommen wird. Nach dem Besuch des Knotens werden die Kinder des Knotens am Ende der Warteschlange eingetragen.
- A – B – C – D – E – F – G





Algorithmus Levelorder Traversierung

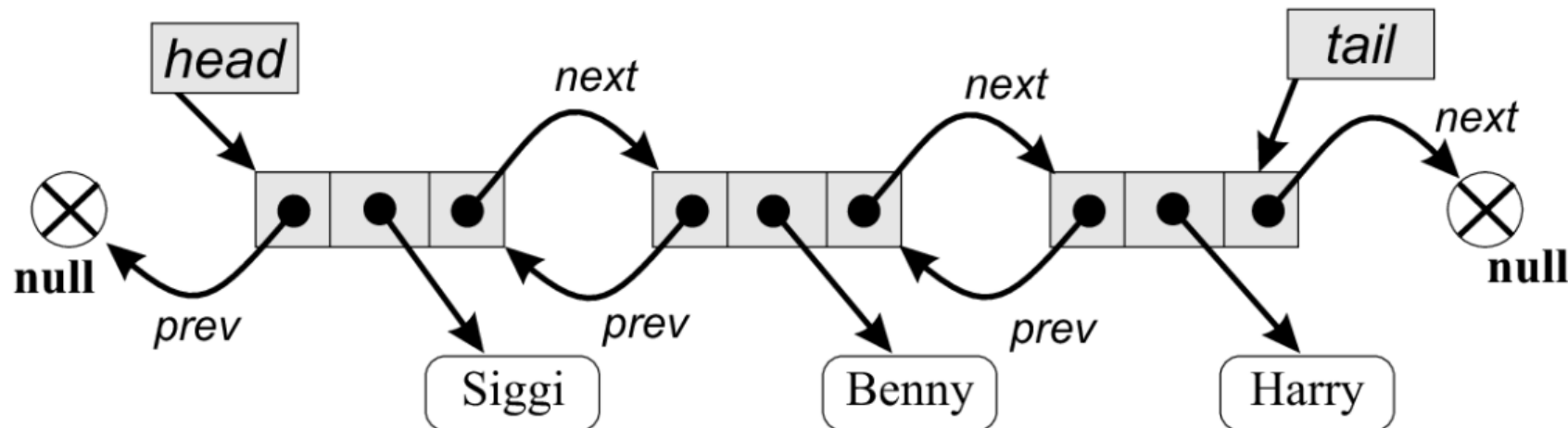
```
algorithm LevelOrder (k)
    Eingabe : Knoten k eines binären Baums
    q := leere Warteschlange;
    q.enter(k); // Wurzel in die Warteschlange aufnehmen
    while not q.isEmpty() do
        Knoten n := q.leave();
        Verarbeite n.elem ;
        q.enter(n.left); // linken Nachfolger eintragen
        q.enter(n.right); // rechten Nachfolger eintragen
    od
```



Doppelt Verkettete Liste

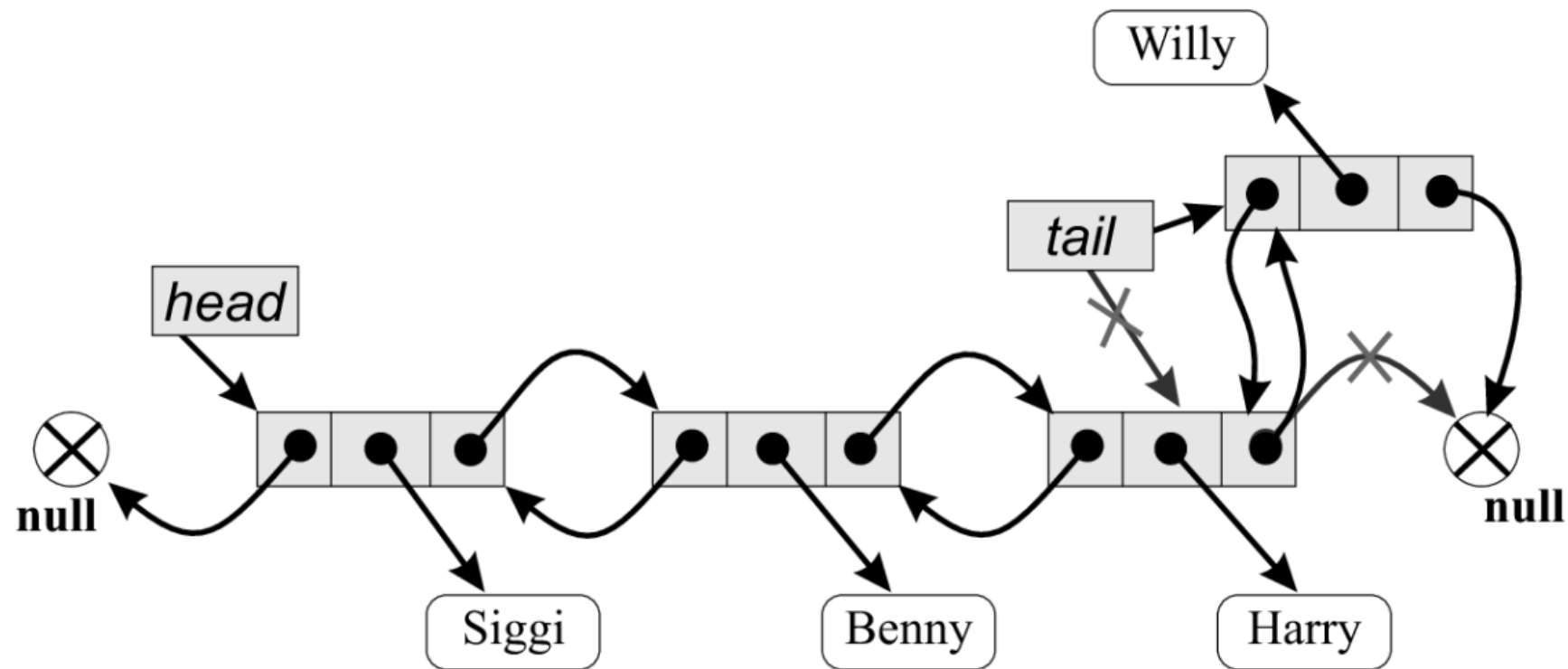
Doppelt verkettete Liste

- einfache Verkettung (nur „Vorwärtszeiger“) erlaubt nur Vorwärtsnavigation
- Rückwärtsnavigation durch zusätzliche „Rückwärtszeiger“: **doppelt verkettete**
- Spezielle Knoten `head`, `tail` für Listenanfang und -ende.





Anhängen am Ende einer doppelt verketteten Liste





Sortiervverfahren